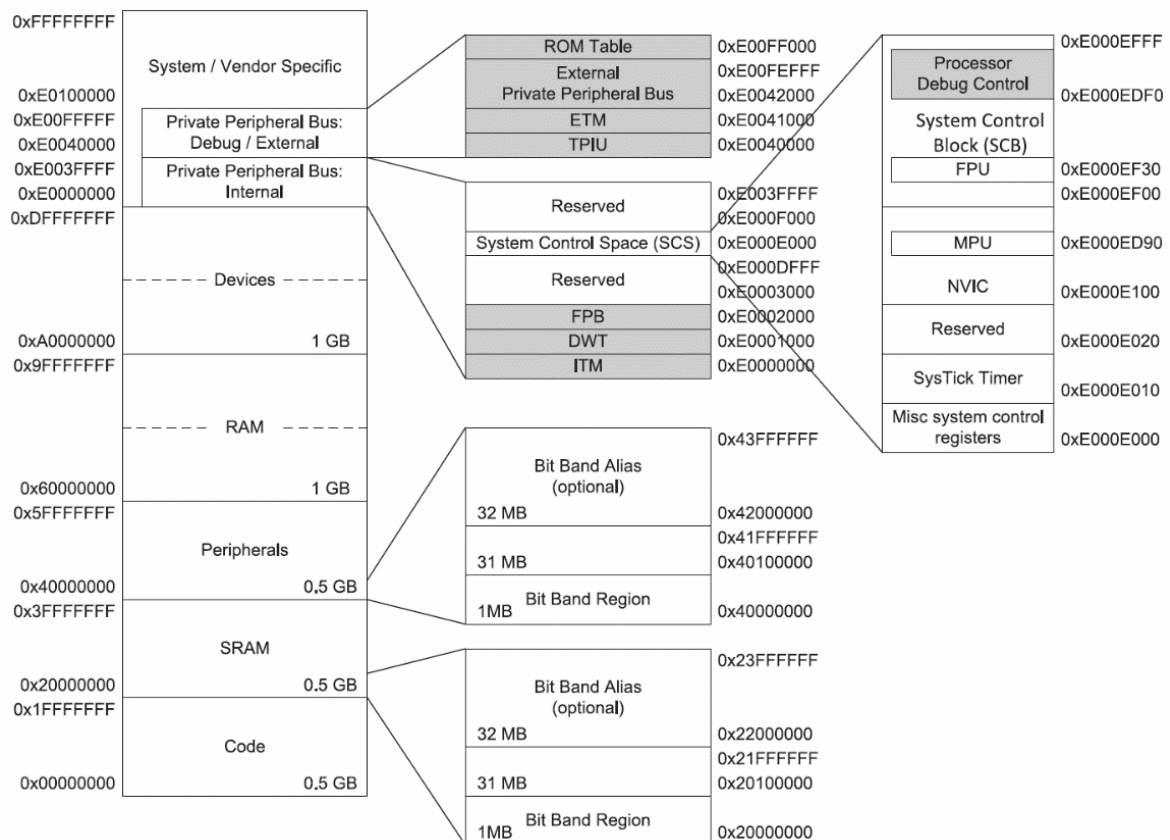


Quick Reference

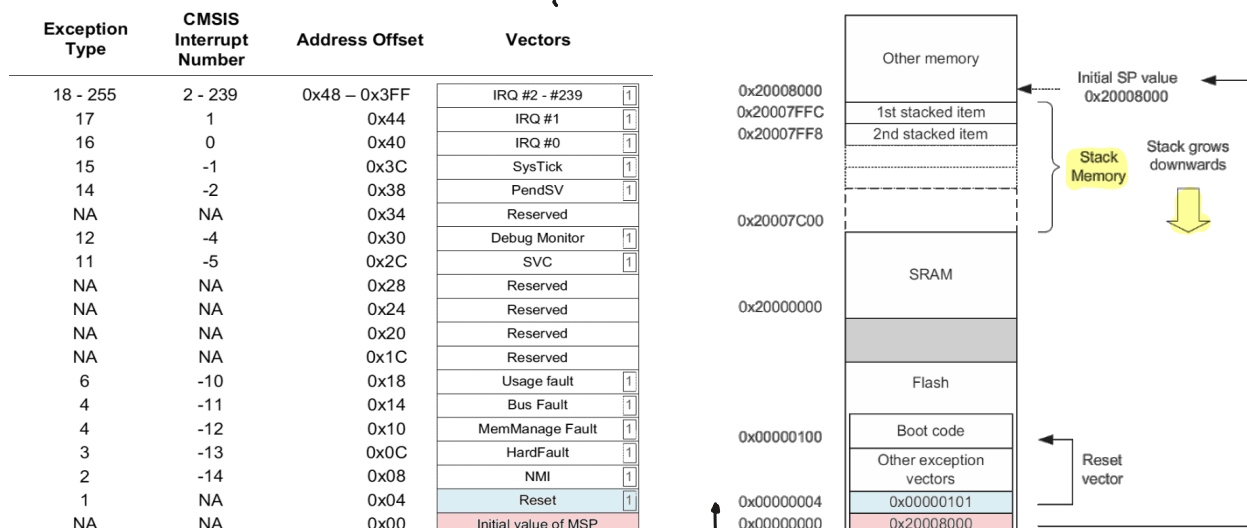
Lars Kamm, Jan Wendler

1 ARM CORTEX-M MEMORY ORGANIZATION

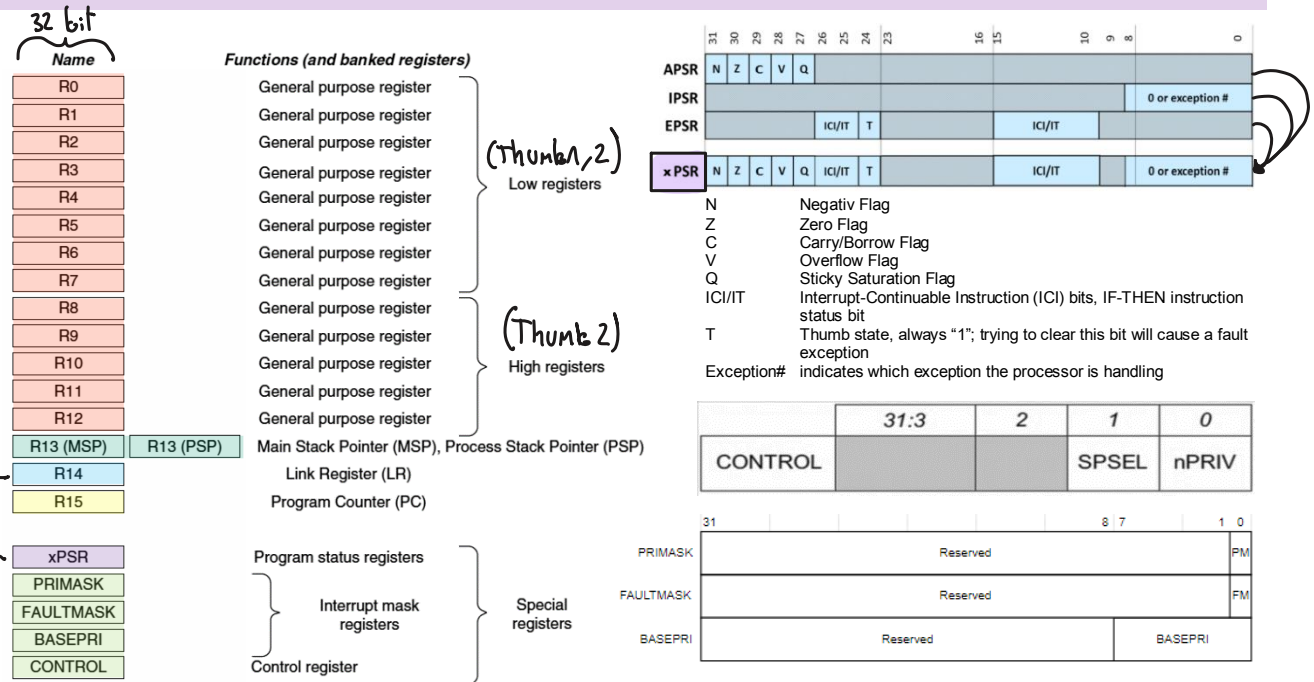
1.1 ARM Cortex-M Memory-Map



1.2 Vector-Table & Reset-Sequence (Process di avvio)

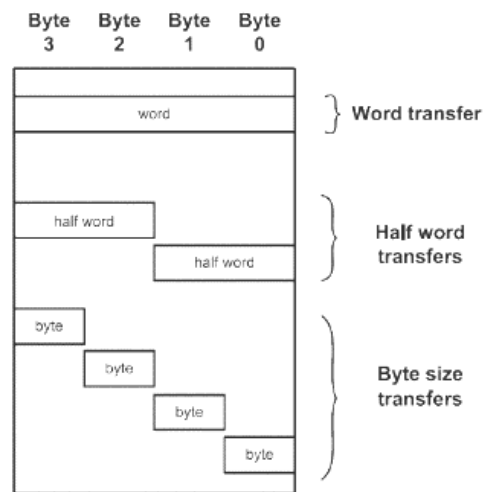
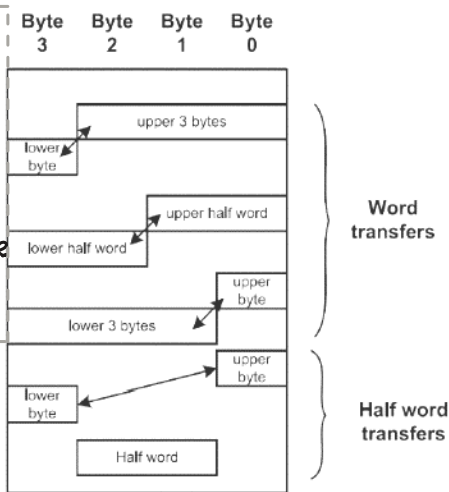


1.3 Register-Set & Program Status Register



1.4 Alignment (Cortex-M => Bus indirizzi di 32 bit (4 byte))

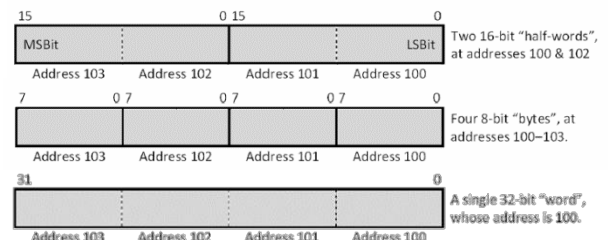
Il processore deve eseguire ⊕ dati a Zugriff leggendo solo specifiche porzioni del bus e ritrappandole correttamente



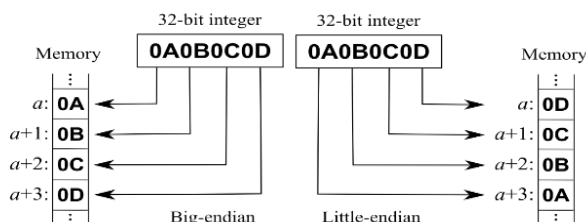
Word (32Bit): 0x0, 0x4, 0x8, 0xC
Accesso solo a multipli di 4

Halfword (16Bit): 0x0, 0x2, 0x4, 0x6, 0x8, 0xA, 0xC, 0xE

Byte (8Bit): 0x0, 0x1, ..., 0xF



1.5 Endianess (Cortex-M => Little-endian)



Endianess betrifft nur Zahlenformate die grösser als ein Byte sind (Halfword, Word, Doubleword).

Little-Endian: Least Significant Byte befindet sich an der tiefsten Adresse.

Big-Endian: Most Significant Byte befindet sich an der tiefsten Adresse.

1.6 Speichergrossen

Zur Angabe der Speicherkapazität als Anzahl von Bits, Bytes oder Codewörtern verwendet man in der Informatik in Anlehnung an die Physik die Bezeichnungen Kilo (K), Mega (M), Giga (G) und Tera (T). Diese beziehen sich auf das Binärzahlensystem:

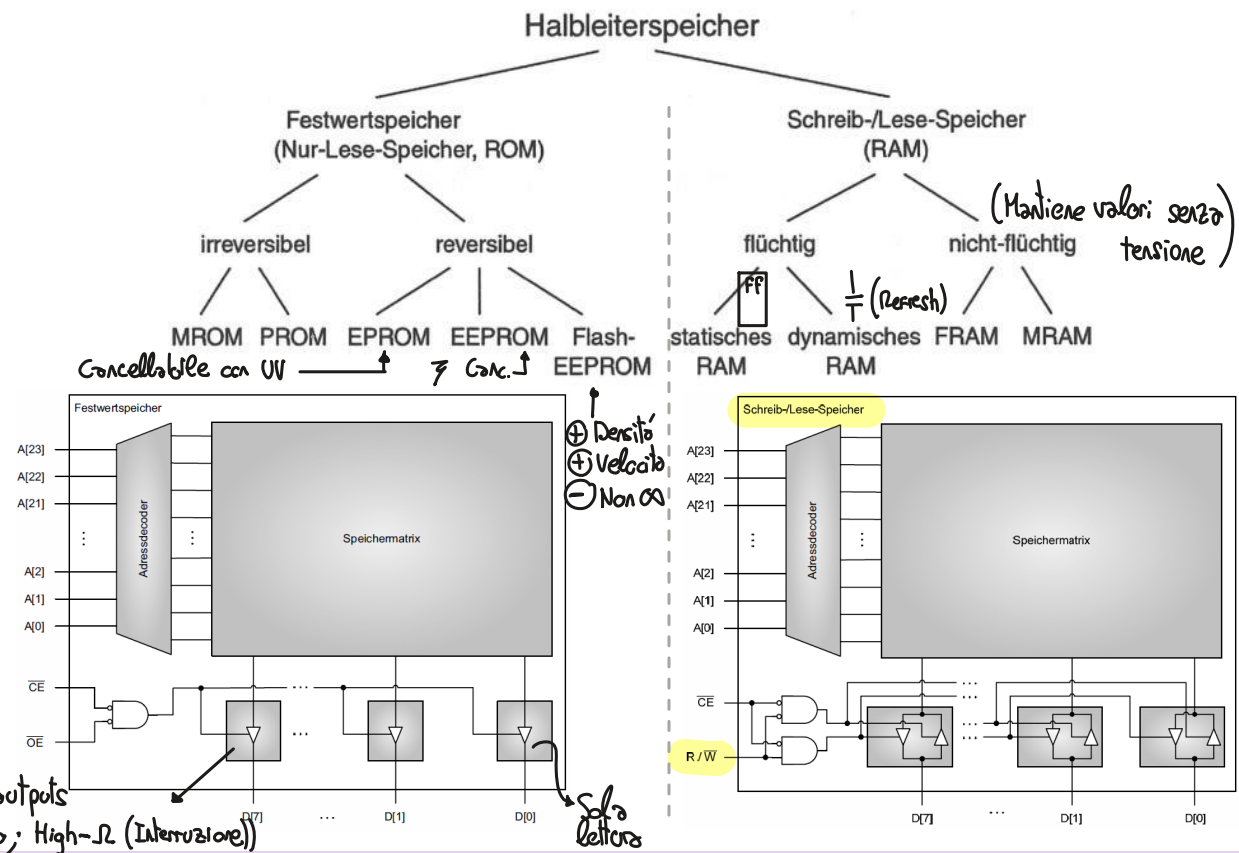
$$1K = 2^{10} = 1'024$$

$$1M = 2^{20} = 1'024K = 1'048'576$$

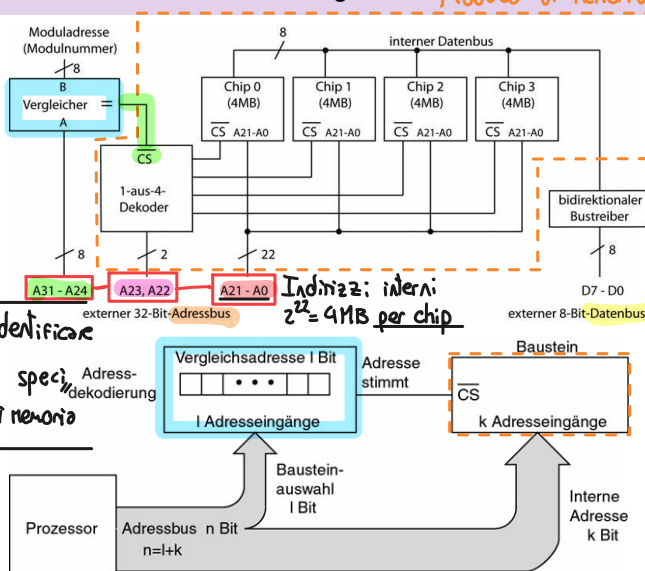
$$1G = 2^{30} = 1'024M = 1'048'576K = 1'073'741'824$$

$$1T = 2^{40} = 1'024G = 1'048'576M = 1'073'741'824K = 1'099'511'627'776$$

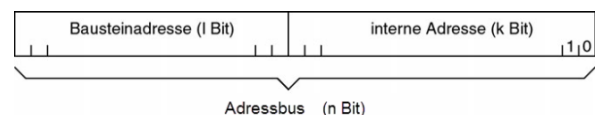
1.7 Speicherarten



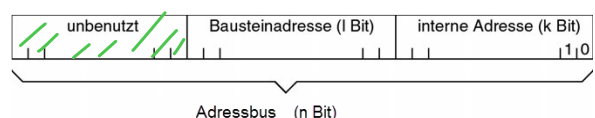
1.8 Adressdekodierung



Esempio $(22+2) + 8 = 32$
 $k + l = n$



$$k + l < n$$

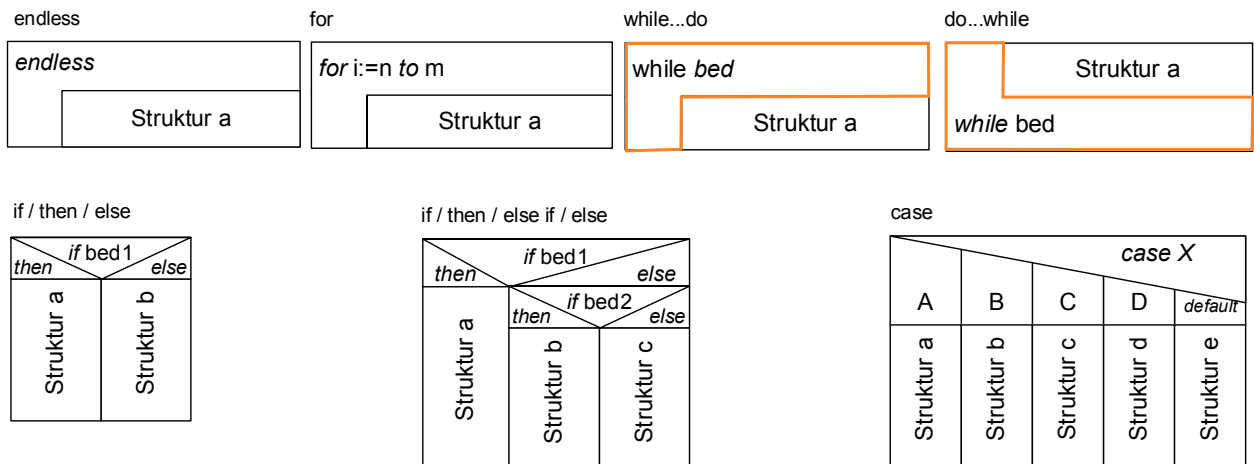


(Chip = contenitore di memoria)

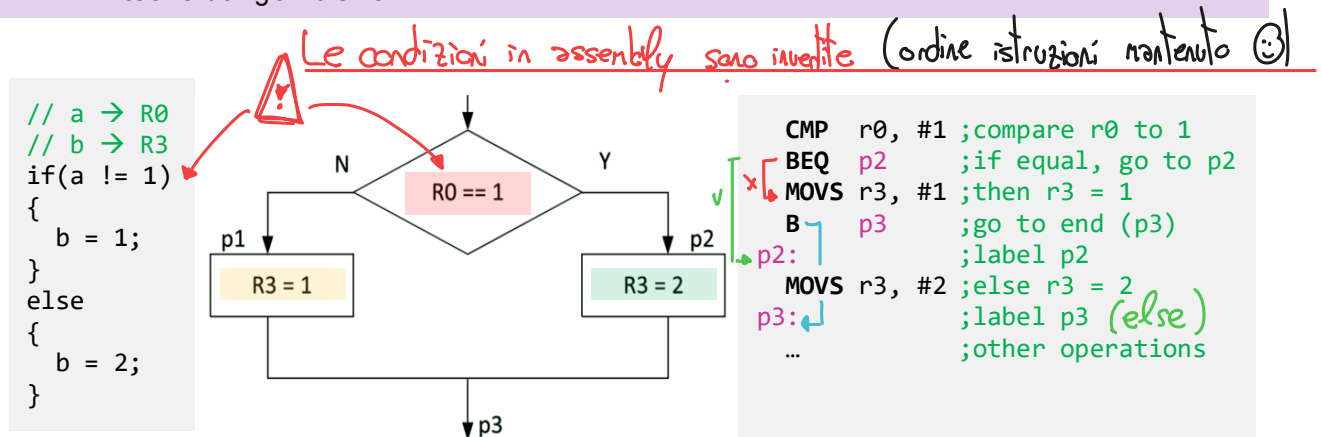
CS = Chip select (accensione chip)

2 KONTROLL- & SELEKTIONSSTRUKTUREN

2.1 Nassi Schneidermann Diagramme



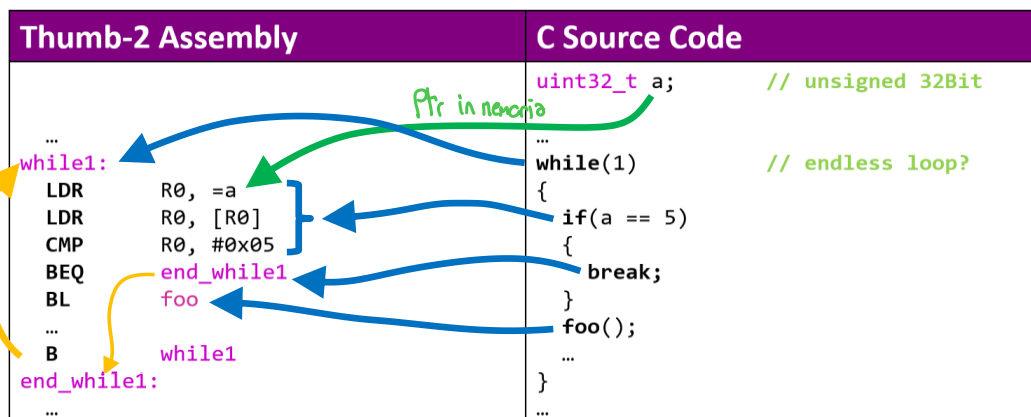
2.2 Entscheidungen treffen



In Assembler-Sprache ist eine Entscheidung praktisch immer ein 2-stufiger Ablauf

- Benötigte Flags ermitteln**
Vergleich (**Compare**) vornehmen, indem zwei Werte voneinander subtrahiert werden. Bei der Differenzbildung ist das Resultat nicht wichtig, sondern nur dessen Eigenschaften (**Flags**).
- Zugehörige Sprünge ausführen**
Im zweiten Schritt werden mit den Flags bedingte Sprünge (**B{cond}**) ausgeführt

2.3 Umsetzung von C/C++ Strukturen *In questo esempio non viene eseguita l'inversione (caso speciale)*



Normalmente viene invertita la cond. per "saltare" subito.
visto che ora tramite break bisogna saltare proprio se 'if'

nei loop la condizione
non è invertita

3 ARM CORTEX-M0(+) INSTRUCTION-SET

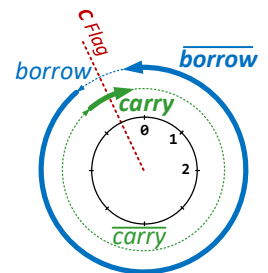
3.1 Symbole & Appendix

Symbols	Meaning
R_d, R_m, R_n	represent 32-bit registers
$\{R_d\}$	if R_d is present R_d is destination, otherwise R_n
$\{S\}$	if S is present, instruction will set condition codes (Aggiornamento flags)
$\{cc\}$	optional logical condition, condition code suffix (EQ, NE, GT, LT, etc.)
$\#imm3$	any value in the range: 0...7
$\#imm5$	any value in the range: 0...31
$\#imm7$	any value in the range: 0..127
$\#imm8$	any value in the range: 0..255
$\#imm10$	any value in the range: 0..1023
$\#imm11$	any value in the range: 0..2047
label	any address within the ROM of the microcontroller; offset range relative to PC: B label -2 KB to +2 KB B<cc> label -256 bytes to +255 bytes BL label -16 MB to +16 MB (32-bit befehl)
$\{reglist\}$	List of registers, sequence is not relevant (Ogni bit della lista indica un registro)

Parentesi grosse: opzionali

3.2 Bedingte Ausführung & Code-Zusätze {cc} (Branches)

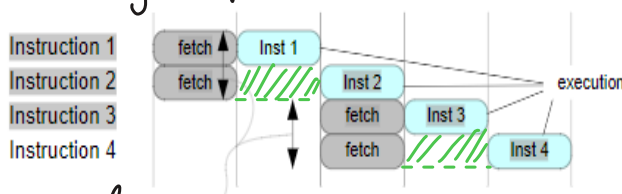
cc	Meaning	Flags (APSR) / Requirements	Binary	uint	int
EQ	Equal	Z = 1	0000	X ==	X ==
NE	Not equal	Z = 0	0001	X !=	X !=
CS or HS	Carry set, Unsigned ≥ carry OR borrow	C = 1	0010	X >=	
CC or LO	Carry clear, Unsigned < carry OR borrow	C = 0	0011	X <	
MI	Minus/negative	N = 1	0100		X -
PL	Positive or zero (non-negative)	N = 0	0101		X +
VS	Overflow	V = 1	0110		X
VC	No overflow	V = 0	0111		X
HI	Unsigned > ("Higher")	C = 1 && Z = 0	1000	X >	
LS	Unsigned ≤ ("Lower or Same")	C = 0 Z = 1	1001	X <=	
GE	Signed ≥ ("Greater than or Equal")	N = V	1010		X >=
LT	Signed < ("Less Than")	N ≠ V	1011		X <
GT	Signed > ("Greater Than")	Z = 0 && N = V	1100		X >
LE	Signed ≤ ("Less than or Equal")	Z = 1 N ≠ V	1101		X <=
AL	Always (unconditional)	any	1110		



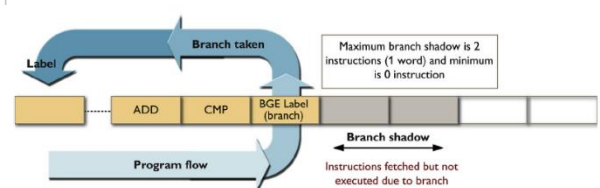
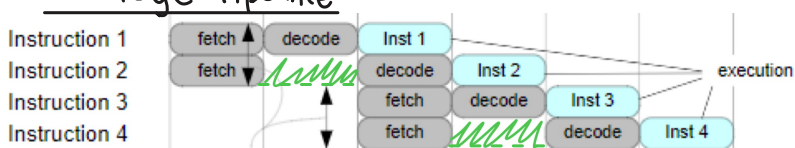
ACHTUNG:
CMP ist eine Subtraktion (Borrow)
CMN ist eine Addition (Carry)

3.3 Pipelining

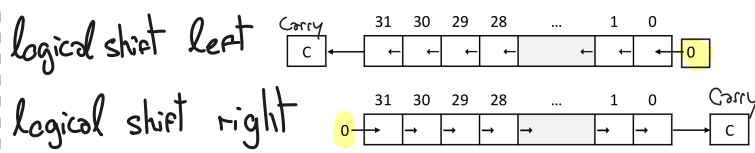
2-Stage Pipeline



3-Stage Pipeline

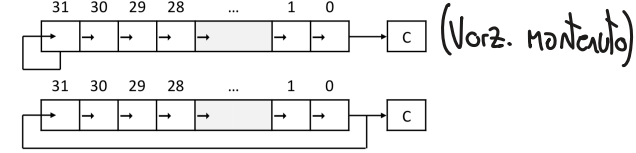


(? → Flag modificata)



Arithmetic Shift right

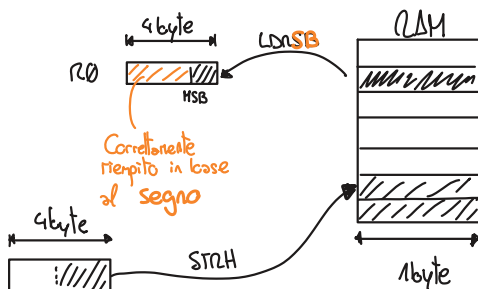
Rotate Shift right



(1) Depends on multiplier implementation
(2) 2 if to AHB interface or SCSS, 1 if to sideband cycle I/O port.

(3) N is the number of elements in the list
(4) N is the number of elements in the list including PC or LR.

Group	Description	Syntax	Cycles	Flags				Instruction Code 16Bit																	
				N	Z	C	V	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Shift (immediate), add, subtract, move, and compare	Move Lo to Lo	$Rd_{0-7} \leftarrow Rm_{0-7}$	MOVS Rd, Rm	1	?	?	-	-	0	0	0	0	0	0	0	0	0	0		Rm			Rd		
	Logical shift left by immediate	$Rd_{0-7} \leftarrow Rm_{0-7} \ll \#imm5$	LSLS Rd, Rm, \#<imm>	1	?	?	?	-	0	0	0	0	0	0					imm5		Rm		Rd		
	Logical shift right by immediate	$Rd_{0-7} \leftarrow Rm_{0-7} \gg \#imm5$	LSRS Rd, Rm, \#<imm>	1	?	?	?	-	0	0	0	0	0	1					imm5		Rm		Rd		
	Arithmetic shift right	$Rd_{0-7} \leftarrow Rm_{0-7} \ggg \#imm5$	ASRS Rd, Rm, \#<imm>	1	?	?	?	-	0	0	0	0	1	0					imm5		Rm		Rd		
	Add 3-bit immediate	$Rd_{0-7} \leftarrow Rm_{0-7} + \#imm3$	ADDS Rd, Rm, \#<imm>	1	?	?	?	?	0	0	0	0	1	1	1	0				imm3		Rm		Rd	
	Add All registers Lo	$Rd_{0-7} \leftarrow Rn_{0-7} + Rm_{0-7}$	ADDS Rd, Rn, Rm	1	?	?	?	?	0	0	0	0	1	1	0	0					Rm		Rn	Rd	
	Sub Lo and Lo	$Rd_{0-7} \leftarrow Rn_{0-7} - Rm_{0-7}$	SUBS Rd, Rn, Rm	1	?	?	?	?	0	0	0	0	1	1	0	1					Rm		Rn	Rd	
	Sub 3-bit immediate	$Rd_{0-7} \leftarrow Rn_{0-7} - \#imm3$	SUBS Rd, Rn, \#<imm>	1	?	?	?	?	0	0	0	0	1	1	1	1				imm3		Rn		Rd	
	Move 8-bit immediate	$Rd_{0-7} \leftarrow \#imm8$	MOV8 Rd, \#<imm>	1	?	?	-	-	0	0	1	0	0	0										imm8	
	Compare Immediate	$Rn_{0-7} - \#imm8 == 0$	CMP Rn, \#<imm>	1	?	?	?	?	0	0	1	0	1											imm8	
Data processing	Add 8-bit immediate	$Rd_{0-7} \leftarrow Rd_{0-7} + \#imm8$	ADDS Rd, \#<imm>	1	?	?	?	?	0	0	1	1	0											imm8	
	Sub 8-bit immediate	$Rd_{0-7} \leftarrow Rd_{0-7} - \#imm8$	SUBS Rd, \#<imm>	1	?	?	?	?	0	0	1	1	1											imm8	
	AND	$Rd_{0-7} \leftarrow Rd_{0-7} \& Rm_{0-7}$	ANDS Rd, Rm	1	?	?	-	-	0	1	0	0	0	0	0	0	0				Rm			Rd	
	Exclusive OR	$Rd_{0-7} \leftarrow Rd_{0-7} \wedge Rm_{0-7}$	EORS Rd, Rm	1	?	?	-	-	0	1	0	0	0	0	0	0	0	1				Rm			Rd
	Logical shift left by register	$Rd_{0-7} \leftarrow Rd_{0-7} \ll Rm_{0-7}$	LSLS Rd, Rm	1	?	?	?	-	0	1	0	0	0	0	0	0	0	1	0			Rm			Rd
	Logical shift right by register	$Rd_{0-7} \leftarrow Rd_{0-7} \gg Rm_{0-7}$	LSRS Rd, Rm	1	?	?	?	-	0	1	0	0	0	0	0	0	0	1	1			Rm			Rd
	Arithmetic shift right by register	$Rd_{0-7} \leftarrow Rd_{0-7} \ggg Rm_{0-7}$	ASRS Rd, Rm	1	?	?	?	-	0	1	0	0	0	0	0	0	1	0	0			Rm			Rd
	Add with carry	$Rd_{0-7} \leftarrow Rd_{0-7} + Rm_{0-7}$	ADCS Rd, Rm	1	?	?	?	?	0	1	0	0	0	0	0	0	1	0	1			Rm			Rd
	Sub with carry	$Rd_{0-7} \leftarrow Rd_{0-7} - Rm_{0-7}$	SBCS Rd, Rm	1	?	?	?	?	0	1	0	0	0	0	0	0	1	1	0			Rm			Rd
	Rotate right by register	$Rd_{0-7} \leftarrow Rd_{0-7} \ggg Rm_{0-7}$	RORS Rd, Rm	1	?	?	?	-	0	1	0	0	0	0	0	0	1	1	1			Rm			Rd
Special data instructions and branch and exchange	AND test	$Rn_{0-7} \& Rm_{0-7}$	TST Rn, Rm	1	?	?	?	-	0	1	0	0	0	0	1	0	0	0				Rm			Rn
	Negate	$Rd_{0-7} \leftarrow \#0 - Rm$	RSBS Rd, Rm, \#0	1	?	?	?	?	0	1	0	0	0	0	1	0	0	1				Rm			Rd
	Compare Lo to Lo	$Rn_{0-7} - Rm_{0-7} == 0$	CMP Rn, Rm	1	?	?	?	?	0	1	0	0	0	0	1	0	1	0				Rm			Rn
	Compare Negative	$Rn_{0-7} - Rm_{0-7} < 0$	CMN Rn, Rm	1	?	?	?	?	0	1	0	0	0	0	1	0	1	1				Rm			Rn
	OR	$Rd_{0-7} \leftarrow Rd_{0-7} Rm_{0-7}$	ORRS Rd, Rm	1	?	?	-	-	0	1	0	0	0	0	1	1	0	0				Rm			Rd
	Multiply	$Rd_{0-7} \leftarrow (Rm_{0-7} * Rd_{0-7}) \gg 32$	MULS Rd, Rm	1 or 32 ⁽¹⁾	?	?	-	-	0	1	0	0	0	0	1	1	0	1				Rm			Rd
	Bit clear	$Rd_{0-7} \leftarrow Rd_{0-7} \& \sim(Rm_{0-7})$	BICS Rd, Rm	1	?	?	-	-	0	1	0	0	0	0	0	1	1	1	0			Rm			Rd
	Move NOT	$Rd_{0-7} \leftarrow \sim(Rm_{0-7})$	MVNS Rd, Rm	1	?	?	-	-	0	1	0	0	0	0	1	1	1	1				Rm			Rd
	Add Any to PC	$PC \leftarrow PC + Rm_{0-15}$	ADD PC, Rm	3	-	-	-	-	0	1	0	0	0	1	0	0	1					Rm			1 1 1
	Add Any to Any	$Rd_{0-15} \leftarrow Rd_{0-15} + Rm_{0-15}$	ADD Rd, Rm	1	-	-	-	-	0	1	0	0	0	1	0	0	0					Rd			Rd
Load/store single data item	Add address from SP and register	$Rd_{0-15} \leftarrow SP + Rd_{0-15}$	ADD Rd, SP	1	-	-	-	-	0	1	0	0	0	1	0	0					Rd			1 1 0 1	
	Compare Any to Any	$Rn_{0-15} - Rm_{0-15} == 0$	CMP Rn, Rm	1	?	?	?	?	0	1	0	0	0	1	0	1					Rn			Rm	
	Move Any to Any	$Rd_{0-15} \leftarrow Rm_{0-15}$	MOV Rd, Rm	1	-	-	-	-	0	1	0	0	0	1	1	0					Rd			Rm	
	Move Any to PC	$PC \leftarrow Rm_{0-15}$	MOV PC, Rm	2	-	-	-	-	0	1	0	0	0	1	1	0	1					Rm			1 1 1
	Branch With exchange	$PC \leftarrow Rm_{0-15}$	BX Rm	2	-	-	-	-	0	1	0	0	0	1	1	1	0					Rm			0 0 0
	Branch With link and exchange	$PC \leftarrow Rm_{0-15}; LR \leftarrow retAddr$	BLX Rm	2	-	-	-	-	0	1	0	0	0	1	1	1	1					Rm			0 0 0
Load from Literal Pool	Load PC-relative	LDR Rd, <label>	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	0	0	1											imm8	
Load/store single data item	Word, register offset	$Rd \rightarrow [Rn + Rm]$	STR Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	0	0	0						Rm			Rn	
	Halfword, register offset	$Rd \rightarrow [Rn + Rm]$	STRH Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	0	0	1						Rm			Rn	
	Byte, register offset	$Rd \rightarrow [Rn + Rm]$	STRB Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	0	1	0						Rm			Rn	
	Signed byte, register offset	$Rd \leftarrow [Rn + Rm]$	LDRSB Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	0	1	0	1					Rm			Rn	
	Word, register offset	$Rd \leftarrow [Rn + Rm]$	LDR Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	1	0	0						Rm			Rn	
	Halfword, register offset	$Rd \leftarrow [Rn + Rm]$	LDRH Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	1	0	1						Rm			Rn	
	Byte, register offset	$Rd \leftarrow [Rn + Rm]$	LDRB Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	1	1	0						Rm			Rn	
	Signed halfword, register offset	$Rd \leftarrow [Rn + Rm]$	LDRSH Rd, [Rn, Rm]	2 or 1 ⁽²⁾	-	-	-	-	0	1	0	1	1	1	1						Rm			Rn	



Load/store

{type}	Data type	Meaning
	32-bit word	0..4'294'967'295 or -2'147'483'648..+2'147'483'647
B	Unsigned 8-bit byte	0..255
SB	Signed 8-bit byte	-128..+127
H	Unsigned 16-bit halfword	0..65'535
SH	Signed 16-bit halfword	-32'768..32'767

Per store non importa pm il valore viene semplicemente tagliato



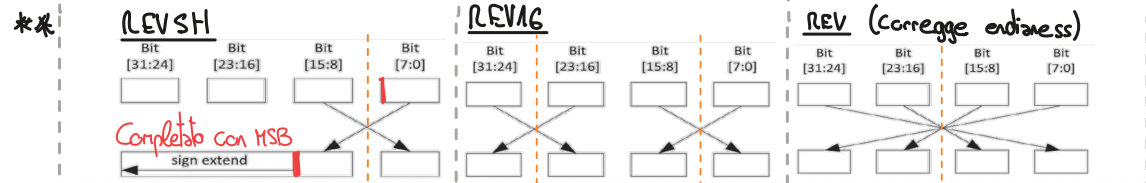
Ostschweizer
Fachhochschule

ComEng I

V2.3

3.4 ARM Cortex-M0(+) Thumb Instructions

```
1 ; Esempio di estensione R0 = 0x55AA8765
2
3 SXTB r1, r0 ; signed extend byte -> r1 = 0x00000065
4 SXTH r1, r0 ; signed extend half-word -> r1 = 0xFFFF8765
5 UXTB r1, r0 ; unsigned extend byte -> r1 = 0x00000065
6 UXTH r1, r0 ; unsigned extend half-word -> r1 = 0x00008765
```



Lars Kamm, Jan Wendler

Quick Reference V2.2

Seite 7 von 12

(5) 2 if taken, 1 if not-taken
(6) Cycle count depends on processor and debug configuration.

(7) Excludes time spent waiting for an interrupt or event.
(8) Executes as NOP.

Load/store single data item	Word, immediate offset	Rd → [Rn + #imm5]	STR Rd, [Rn, #<imm>]	2 or 1 ⁽⁴⁾	-	-	-	-	0	1	1	0	0	imm5	Rn	Rd
	Word, immediate offset	Rd ← [Rn + #imm5]	LDR Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	-	-	-	-	0	1	1	0	1	imm5	Rn	Rd
	Byte, immediate offset	Rd → [Rn + #imm5]	STRB Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	-	-	-	-	0	1	1	1	0	imm5	Rn	Rd
	Byte, immediate offset	Rd ← [Rn + #imm5]	LDRB Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	-	-	-	-	0	1	1	1	1	imm5	Rn	Rd
	Halfword, immediate offset	Rd → [Rn + #imm5]	STRH Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	-	-	-	-	1	0	0	0	0	imm5	Rn	Rd
	Halfword, immediate offset	Rd ← [Rn + #imm5]	LDRH Rd, [Rn, #<imm>]	2 or 1 ⁽²⁾	-	-	-	-	1	0	0	0	1	imm5	Rn	Rd
Generate PC-relative address	SP-relative	Rd → [SP + #imm8]	STR Rd, [SP, #<imm>]	2 or 1 ⁽²⁾	-	-	-	-	1	0	0	1	0	Rd	imm8	
	SP-relative	Rd ← [SP + #imm8]	LDR Rd, [SP, #<imm>]	2 or 1 ⁽²⁾	-	-	-	-	1	0	0	1	1	Rd	imm8	
Generate SP-relative address	Form address from PC	Rd ← addr[<label>]	ADR Rd, <label>	1	-	-	-	-	1	0	1	0	0	Rd	imm8	
	Form address from SP	Rd ← SP + #imm8	ADD Rd, SP, #<imm>	1	-	-	-	-	1	0	1	0	1	Rd	imm8	
Miscellaneous 16-bit instructions	Add Immediate to SP	SP ← SP + #imm7	ADD SP, #<imm>	1	-	-	-	-	1	0	1	1	0	0	0	0
	Sub Immediate from SP	SP ← SP - #imm7	SUB SP, #<imm>	1	-	-	-	-	1	0	1	1	0	0	0	1
	Extend signed halfword to word	Rd _{32Bit} ← signed Rm _{16Bit}	SXTH Rd, Rm	1	-	-	-	-	1	0	1	1	0	0	1	0
	Extend signed byte to word	Rd _{32Bit} ← signed Rm _{8Bit}	SXTB Rd, Rm	1	-	-	-	-	1	0	1	1	0	0	1	0
	Extend unsigned halfword	Rd _{32Bit} ← Rm _{16Bit}	UXTB Rd, Rm	1	-	-	-	-	1	0	1	1	0	0	1	0
	Extend unsigned byte	Rd _{32Bit} ← Rm _{8Bit}	UXTB Rd, Rm	1	-	-	-	-	1	0	1	1	0	0	1	1
Miscellaneous 16-bit instructions	Push	regs → mem[SP--]	PUSH {<LoReglist>}	1+N ⁽⁴⁾	-	-	-	-	1	0	1	1	0	1	0	0
	Push with link register	regs, LR → mem[SP--]	PUSH {<LoReglist>, LR}	1+N ⁽³⁾	-	-	-	-	1	0	1	1	0	1	0	1
	Disable interrupts		CPSID i	1	-	-	-	-	1	0	1	1	0	1	1	0
	Enable interrupts		CPSIE i	1	-	-	-	-	1	0	1	1	0	1	1	0
	Bytes in word	Rd ← RevByteOrder[Rm]	REV Rd, Rm	1	-	-	-	-	1	0	1	1	1	0	0	0
	Bytes in both halfwords	Rd ← RevHalfWords[Rm]	REV16 Rd, Rm	1	-	-	-	-	1	0	1	1	1	0	0	1
Miscellaneous 16-bit instructions	Signed bottom half word	Rd ← RevLoHalf[Rm]	REVSH Rd, Rm	1	-	-	-	-	1	0	1	1	1	0	0	1
	Pop	regs ← mem[SP++]	POP {<LoReglist>}	1+N ⁽³⁾	-	-	-	-	1	0	1	1	1	1	0	0
	Pop and return	regs, PC ← mem[SP++]	POP {<LoReglist>, PC}	3+N ⁽⁴⁾	-	-	-	-	1	0	1	1	1	1	0	1
	Breakpoint		BKPT #<imm>	- ⁽⁶⁾	-	-	-	-	1	0	1	1	1	1	1	0
	Send event		SEV	1	-	-	-	-	1	0	1	1	1	1	1	1
	Wait for event		WFE	2 ⁽⁷⁾	-	-	-	-	1	0	1	1	1	1	1	1
Store multiple registers	Wait for interrupt		WFI	2 ⁽⁷⁾	-	-	-	-	1	0	1	1	1	1	1	1
	Yield		YIELD	1 ⁽⁸⁾	-	-	-	-	1	0	1	1	1	1	1	1
	No operation		NOP	1	-	-	-	-	1	0	1	1	1	1	1	1
	Store Multiple	regs → mem[Rn++]	STM Rn!, {<LoReglist>}	1+N ⁽³⁾	-	-	-	-	1	1	0	0	0	Rn	lo_register_list	
	Load Multiple, excluding base	regs ← mem[Rn++]	LDM Rn!, {<LoReglist>}	1+N ⁽³⁾	-	-	-	-	1	1	0	0	1	Rn	lo_register_list	
	Load Multiple, including base	regs ← mem[Rn]	LDM Rn, {<LoReglist>}	1+N ⁽³⁾	-	-	-	-	1	1	0	0	1	Rn	lo_register_list	
Conditional branch, and Supervisor Call	Conditional (BEQ, ...)	if <cc> then PC ← label	B<cc> <label>	1 or 2 ⁽⁵⁾	-	-	-	-	1	1	0	1	cond	imm8		
	Supervisor Call		SVC #<imm>	- ⁽⁶⁾	-	-	-	-	1	1	0	1	1	1	1	1
	Unconditional Branch	PC ← label	B<label>	2	-	-	-	-	1	1	1	0	0	imm11		
	Branch & Link 32Bit	With link	BL<label>	3	-	-	-	-	1	1	1	1	0	S	imm10	
		[15 - 0]			-	-	-	-	1	1	1	1	1	J2	imm11	
		[31 - 16]			-	-	-	-	1	1	1	1	1	J2	imm11	
Write and Read Special Register 32 Bit	Read special register	[15 - 0]	MRS Rd, <specreg>	3	-	-	-	-	1	1	1	1	0	0	1	1
		[31 - 16]			-	-	-	-	1	0	0	0	Rd	SYSm		
	Write special register	[15 - 0]	MSR <specreg>, Rn	3	-	-	-	-	1	1	1	1	0	0	1	1
		[31 - 16]			-	-	-	-	1	0	0	0	1	0	0	0
					-	-	-	-	1	0	0	0	1	0	0	0
					-	-	-	-	1	0	0	0	1	0	0	0
Synchronisation 32 Bit	Instruction synchronization barrier	[15 - 0]	ISB	3	-	-	-	-	1	1	1	1	0	0	1	1
		[31 - 16]			-	-	-	-	1	0	0	0	1	1	1	1
	Data memory barrier	[15 - 0]	DMB	3	-	-	-	-	1	1	1	1	0	0	1	1
		[31 - 16]			-	-	-	-	1	0	0	0	1	1	1	1
					-	-	-	-	1	0	0	0	1	1	1	1
	Data synchronization barrier	[15 - 0]	DSB	3	-	-	-	-	1	1	1	1	0	0	1	1
Synchronisation 32 Bit		[31 - 16]			-	-	-	-	1	0	0	0	1	1	1	1
					-	-	-	-	1	0	0	0	1	1	1	1
					-	-	-	-	1	0	0	0	1	1	1	1
					-	-	-	-	1	0	0	0	1	1	1	1
					-	-	-	-	1	0	0	0	1	1	1	1
					-	-	-	-	1	0	0	0	1	1	1	1



Ostschweizer
Fachhochschule

ComEng I
V2.3

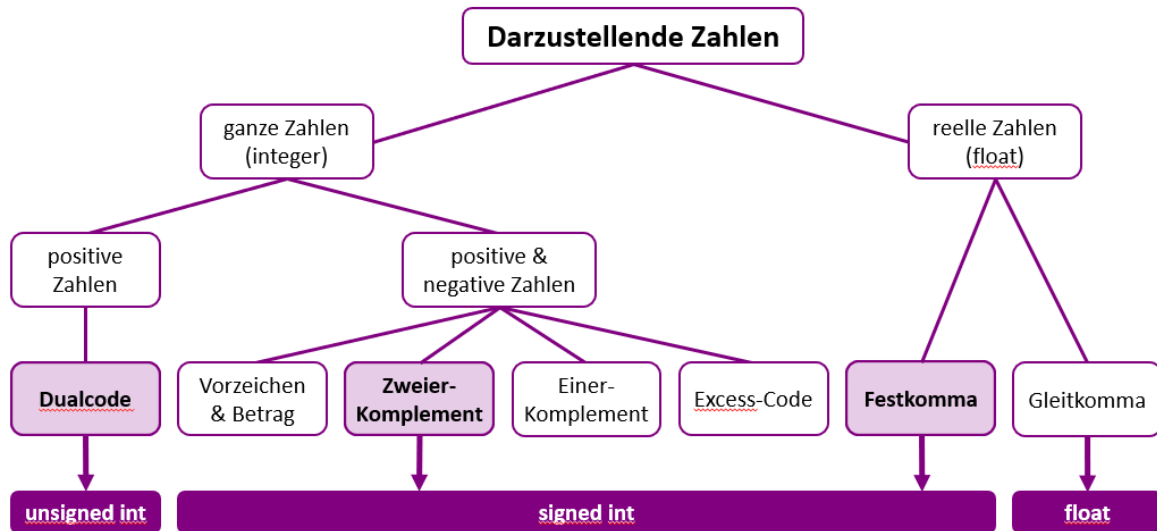
Mehrfache Datentransfers

- il '!' serve per aggiornare l'indirizzo Rn (dopo punter  a primo spazio libero dopo array)
- STM solo versione con !

- BL → salto all'indirizzo specificato e salva istruzione dopo attuale in LR (R14). Per poter tornare indietro a fine fn
- BX Rn → Salto all'indirizzo contenuto in un registro (solitamente LR)
- CMN → Viene usato quando il secondo termine   negativo (a == -5 / a > -3)

Immediate verranno salvati su reg.

4 ZAHLENSYSTEME



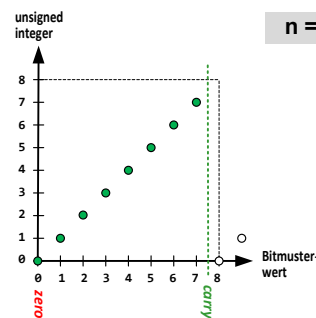
4.1 Vorzeichenlose Zahlen

Dualcode (unsigned int)

$$W = \sum_{i=0}^{n-1} d_i \cdot 2^i$$

$d_{n-1} \quad d_{n-2} \quad \dots \quad d_2 \quad d_1 \quad d_0$

Relevante Flags: Zero, Carry/Borrow



Bereich
 $0 \dots 2^n - 1$

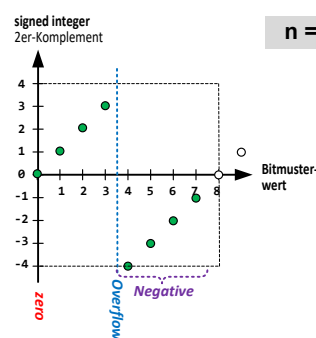
4.2 Vorzeichenbehaftete Zahlen

Zweierkomplement (signed int) → Komponente ALU ⊕ unsigned

$$W = \left(\sum_{i=0}^{n-2} d_i \cdot 2^i \right) - d_{n-1} \cdot 2^{n-1}$$

$d_{n-1} \quad d_{n-2} \quad \dots \quad d_2 \quad d_1 \quad d_0$

Relevante Flag: Zero, Negativ, Overflow
MSB = 1



Bereich
 $-2^{n-1}, \dots, 2^{n-1} - 1$

4.3 Zweierkomplement-Negierung

Vorgehensweise:

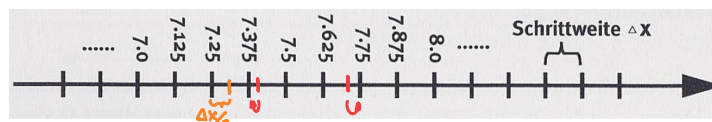
- 1) Vorzeichenbehaftete Zahl in Binärcode darstellen
- 2) Alle Bits einzeln invertieren
- 3) Binäre Addition mit dem Wert +1
- 4) Resultat entspricht der negierten Vorzeichenbehaftete Zahl

Beispiel: $-3_{10} = 101_2$
Invertieren: 010_2
+1: 001_2

 011_2
=====

Resultat: $+3_{10}$

$$d_n \cdot 2^n + d_{n-1} \cdot 2^{n-1} + \dots + d_0 \cdot 2^0$$



4.4 Festkomma-Zahlen

Bisogna correggere i numeri (Nearest) !

Dec. Bin.
Da 10 a 0b calcolare 1 passo in
nella parte dopo virgola e vedere se serve
rounding

Vorzeichenlose Fixed-Point-Zahlen

Allgemeine Formel (Dualcode als Basis)

$$W = \sum_{i=-m}^{n-1} d_i \cdot 2^i$$

$d_{n-1} d_{n-2} \dots d_1 d_0, d_{-1} d_{-2} d_{-m}$

Wertebereich: $0.0 \dots 2^n - 2^{-m}$

Schrittweite: $\Delta X = 2^{-m}$

Max. Diskretisierungsfehler: $\frac{\Delta X}{2} = 2^{-(m+1)}$

Vorzeichenbehaftete Fixed-Point-Zahlen

Allgemeine Formel (2er-Komplement als Basis)

$$W = \left(\sum_{i=-m}^{n-2} d_i \cdot 2^i \right) - d_{n-1} \cdot 2^{n-1}$$

$d_{n-1} d_{n-2} \dots d_1 d_0, d_{-1} d_{-2} d_{-m}$

Wertebereich: $-2^{n-1} \dots 2^{n-1} - 2^{-m}$

Schrittweite: $\Delta X = 2^{-m}$

Max. Diskretisierungsfehler: $\frac{\Delta X}{2} = 2^{-(m+1)}$

4.5 IQ-Zahlenformate

Immer signed !

Integer/Quotient-Darstellung

Signed Fixed-Point mit $n = I$ und $m = Q$.
Numero cifre decim. →
Numero cifre dopo la virgola

$$W = -d_{I-1} \cdot 2^{I-1} + \sum_{j=0}^{I-2} d_j \cdot 2^j + \sum_{i=1}^Q d_{-i} \cdot 2^{-i}$$

$d_{I-1} d_{I-2} \dots d_1 d_0, d_{-1} d_{-2} d_{-Q}$

Werteb. $-2^{I-1} \dots 2^{I-1} - 2^{-Q}$

Fractionale Zahlen (oder Spezialfall IQx)

$$W = -d_0 + \sum_{i=1}^Q d_{-i} \cdot 2^{-i}$$

$d_0, d_{-1} d_{-2} \dots d_{-Q}$

Wertebereich: $-1.0 \leq W < +1.0$

Vorzeichen: Koeffizient d_0

Genauigkeit: $2^{-(Q)}$

⊕ Overflow solo quando eseguo $-1 + -1$ (Weniger Aufwand)

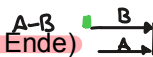
4.6 Zahlenkreise

Addition: Zahlenvektoren hintereinander reihen (Anfang auf Ende)

$A+B$



Subtraktion: Zahlenvektoren stumpf gegeneinanderstellen (Ende auf Ende)



Bsp.

$0x2 - 0x8$

Unsigned Integer

Relevante Flags: Zero, Carry/Borrow

Zahlenvektoren immer Uhrzeigersinn

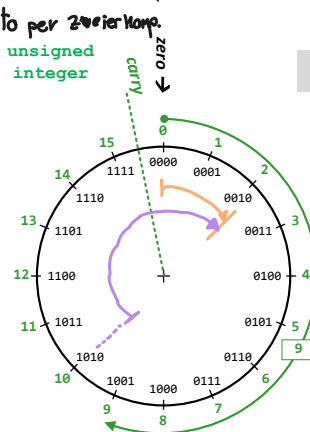
Signed Integer

(Primo bit 1) $(C_n \neq C_{n-1})$ (0...0)

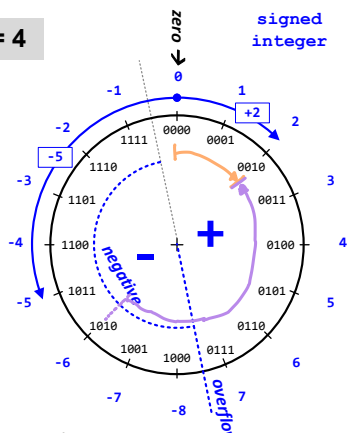
Relevante Flags: Negative, Overflow, Zero

positiv → Zahlenvektor Uhrzeigersinn

negativ → Zahlenvektor Gegenuhrzeiger



$n = 4$



□ = Corrispondente carry nella sottrazione
(Comportamento invertito rispetto a ⊕)

$\begin{cases} 1 = \text{risultato} \geq 0 \\ 0 = \text{risultato} < 0 \end{cases}$ (il risultato è corretto)

Nella ruota **UNSIGNED** tutti i vettori vanno in senso orario

Nella ruota **SIGNED** i vettori ≥ 0 in senso orario, < 0 in senso antiorario (regola ⇒ rimane)

Carry

Addizione: se $C_n = 1 \rightarrow \text{Carry} = 1$

Sottrazione: se $C_n = 0 \rightarrow \text{Carry} = 1$ (Negato)

Bsp

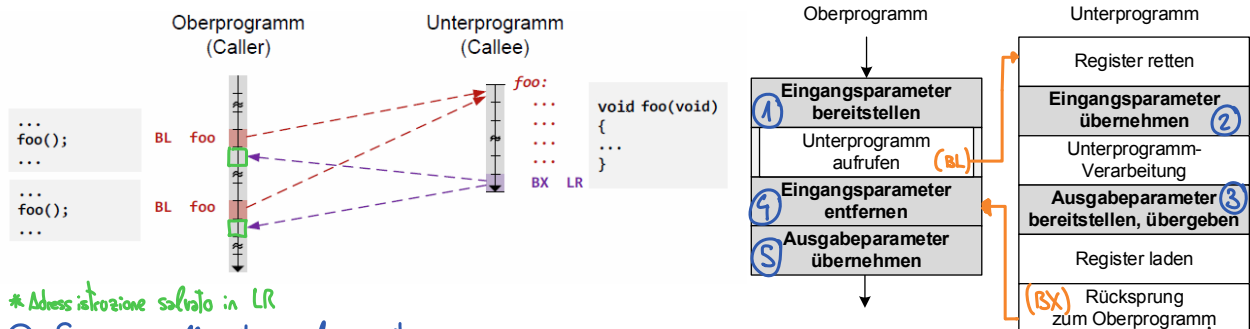
$0x2 - 0x8 \rightarrow$

$\begin{array}{r} 0010 \\ - 1000 \\ \hline 1010 \end{array}$

$\begin{array}{l} Z = 0 \\ C = 0 \\ N = 1 \\ V = 1 \end{array}$

* Il compilatore provvede allo stack frame
LR e ulteriori registri per il calcolo, se necessario

5 SUBROUTINEN

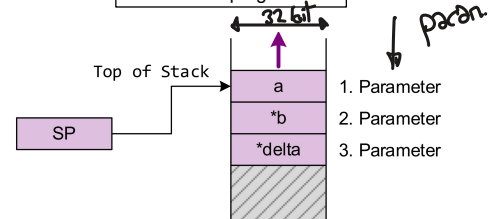


* Indirizzo salvato in LR

④ Se sono sullo stack, eliminare stack frame

* Regeln zum ARM Architecture Procedure Call Standard (AAPCS)

- Die Register **R0-R3** werden als Parameter verwendet.
- Weitere Parameter werden auf den **Stack** gelegt. (> 4 Parameter)
- Der **Rückgabewert** ($\leq 32\text{Bit}$) wird in Register **R0** übertragen.
- Ein **64Bit-Rückgabewert** wird in den Registern **R0** und **R1** übertragen.
- Funktionen müssen die Registerinhalte **R4-R11** während der Ausführung sichern und rekonstruieren.
- Die Register **R0-R3** und **R12** dürfen in der Subroutine ohne Sicherung verändert werden.
- Es wird stets ein **8-Byte Alignment** auf dem Stack eingehalten.



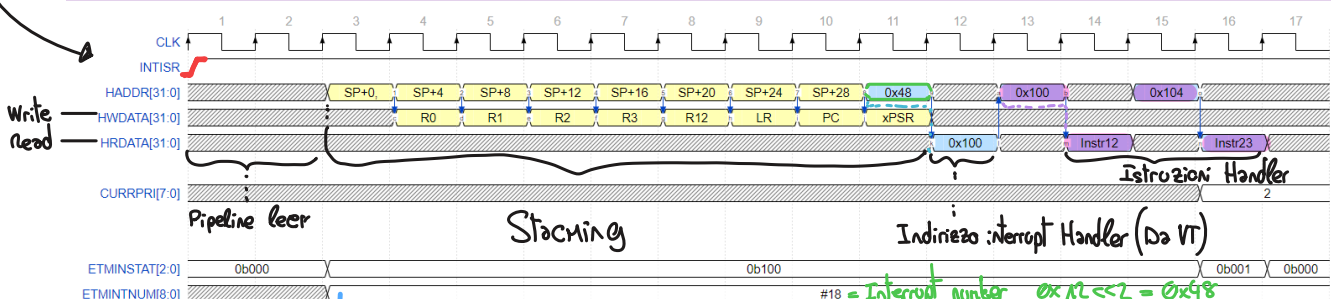
Tutti i parametri ($\leq 32\text{byte}$) vengono sempre passati come se fossero di 32 bit (int, float, double, etc.)

Register	Verwendungszweck	Scratch	Sichern
R0	Erster Funktionsparameter, Return-Wert (8-/16-/32-Bit)	Ja	
R1	Zweiter Funktionsparameter, Upper Word Return-Wert (64-Bit)	Ja	
R2, R3	Dritter und vierter Funktionsparameter	Ja	
R4-R7	Registervariablen (temporär)	Ja	Ja
R8-R11	Compilerabhängige Verwendung	Ja	Ja
R12	Funktionsinternes temporäres Speicherregister für Sprünge	Ja	
R13	Stack-Pointer (SP, Top-of-Stack)	Diese Register werden für spezielle Zwecke verwendet	
R14	Link Register (LR)		
R15	Programm Counter (PC)		

Bus HADDR é sempre 1 ciclo in anticipo rispetto a H...DATA (Address Fetching)

6 INTERRUPTS & EXCEPTIONS

6.1 Vector Fetch & Stacking



Fasi Interrupt

- Request:** (Arriva IRQ)
- Stacking & Vector-Fetch:** Register auf Stack sichern und Sprung via Vektortabelle ausführen und Active Status setzen.
- ISR Handler:** Benutzercode ausführen und am Ende den Active Status zurücksetzen
- Unstacking:** Register wieder herstellen, Stack abbauen und Rücksprung mittels LR

R0-R12, LR, PC, xPSR automaticamente salvati

1 Hex → Dez

Hex	A	B	C	D	E	F
Dec	10	11	12	13	14	15
Bin	1010	1011	1100	1101	1110	1111

2 Instruction Set Architecture (ISA)

L'evoluzione delle architetture ARM ha portato alla coesistenza di diversi set di istruzioni.

- **ARM (32-bit) (Obsoleto):** 32 bit, massime prestazioni ma occupano maggior spazio in memoria.
- **Thumb (16-bit):** sottoinsieme 16 bit, Ideale per ridurre il footprint della memoria (circa 30% di risparmio)
- **Thumb-2 (16/32-bit):** lunghezza variabile (16 e 32 bit). Permette di ottenere una densità di codice simile a Thumb con prestazioni analoghe al set ARM a 32 bit.

3 Logica della ALU e Program Status Register

L'Unità Aritmetico-Logica (ALU) opera esclusivamente su stringhe binarie di bit, ignorando la semantica del segno (interpretazione *signed* vs *unsigned*).

3.1 Meccanismo delle Flag (APSR)

- **N (Negative):** Riflette il bit più significativo (MSB) del risultato. Se $MSB = 1$, la flag è impostata a 1.
- **Z (Zero):** Impostata a 1 se tutti i bit del risultato sono pari a zero.
- **C (Carry):** Indica un riporto oltre il bit più significativo.
 - Fondamentale per operazioni *unsigned*. **Invertito per la sottrazione!**
 - **Meccanismo *sottrazione*:** $(A + NOT(B) + 1)$. Zweierkompl.
 - * $C = 1 \implies A \geq B$ (Nessun prestito / Not-Borrow).
 - * $C = 0 \implies A < B$ (Prestito avvenuto / Borrow).
- **V (oVerflow):** Indica un errore di segno nel calcolo in complemento a due, Fondamentale per operazioni *signed*.
 - $(+) + (+) = (-)$ Impossibile
 - $(-) + (-) = (+)$ Impossibile

3.1.1 Interpretazione del Segno e Salti Condizionati

Poiché la ALU aggiorna simultaneamente sia la flag *C* che la flag *V*, la distinzione tra variabile *int* e *uint* non risiede nell'hardware, ma nel software.

4 Componenti del Sistema (SCS)

Il **System Control Space (SCS)** gestisce le funzionalità core del processore, inclusa la configurazione del NVIC e del registro di controllo di sistema.

4.1 Meccanismi di Fault e Eccezioni

- **HardFaults:** Sempre attivati; gestiscono errori di Bus, Usage e Memory Management. Permettono una reazione rapida agli errori critici.
- **Wake-up Interrupt:** Consente la disattivazione gerarchica dei componenti per ridurre il consumo energetico (modalità Sleep/Stop).
- **Memory Protection Unit (MPU):** Genera eccezioni in caso di accessi non autorizzati alla memoria.
- **Debug Unit:** Fornisce fino a 4 breakpoint hardware per il debugging del codice.

4.2 System Tick Timer (SysTick)

Un contatore a **24-bit** che fornisce una base dei tempi costante per il sistema operativo o i cicli di programma.

5 Stack e Gestione Memoria

Lo **Stack** è una memoria temporanea utilizzata per variabili locali e salvataggio contestuale durante le eccezioni.

- **Direzione:** Cresce verso indirizzi di memoria inferiori.
- **Stack Pointer (SP):** Punta sempre al "top of stack". È vietato leggere o scrivere al di sotto di SP.
- **Allineamento:** Normalmente allocato in step di 8 byte.
- **Velocità:** circa due volte più rapido del RAM/ROM pointer poiché (SP) è già disponibile.

6 Gestione degli Interrupt (NVIC)

- Il **Nested Vector Interrupt Controller** gestisce la prioritizzazione e il passaggio tra il programma di background e le **Interrupt Service Routine (ISR)**.
- **L'LSB in ogni vettore indica la modalità di salto (1: thumb, 0: ARM) nei Cortex-M sempre 1** (Anche indirizzi in LR con B. .)
- Solitamente la VT (vector table) viene salvata in memoria fra il boot code e la seconda istruzione.
- Tramite VTOR (Vector Table Offset Register) è possibile cambiare la posizione della tabella in memoria.
- **Stati dell'Interrupt:** *Pending* (richiesto dalla periferica, azzerato automaticamente dall'hardware all'ingresso della ISR), *Active* (in esecuzione in Handler Mode), *Inactive*.

6.1 NVIC Special Features

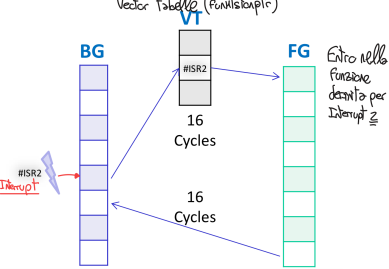
- **Tail-Chaining:** IRQ a bassa priorità è pendente al termine di ISR attiva, la CPU salta l'Unstacking e lo Stacking. Fa solo un nuovo Vector Fetch (→ transizione di soli **6 cicli**).
- **Late Arrival:** IRQ ad alta priorità si attiva durante lo stacking di un IRQ a bassa priorità, la CPU completa stacking ma esegue direttamente Vector Fetch dell'IRQ più urgente.
- **Pop-Preemption:** IRQ arriva durante l'Unstacking (Pop) finale, l'hardware abbandona il Pop e devia subito alla nuova ISR in soli **6 cicli**.

6.2 Registri di Mascheramento

- **PRIMASK:** 1 bit. Disabilita tutti gli interrupt programmabili (tranne HardFault e NMI).
- **FAULTMASK:** 1 bit. Disabilita interrupt e tutte le eccezioni (tranne NMI).
- **BASEPRI:** Disabilita interrupt con priorità numerica maggiore o uguale (urgenza inferiore) a un valore impostato.

6.3 Latenza e Stacking

- **Interrupt Latency:** Tempo fisso d'ingresso nella ISR senza overhead software. **Cortex-M0+:** 15–16 cicli di clock. **Cortex-M3/M4:** 12 cicli.
- Le funzioni ISR sono di tipo void **ISR_Handler(void):** non accettano parametri e non restituiscono valori.

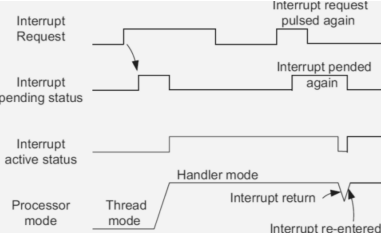
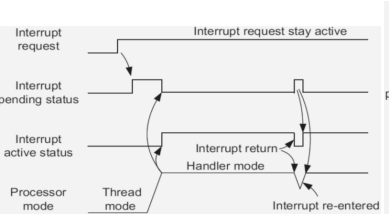
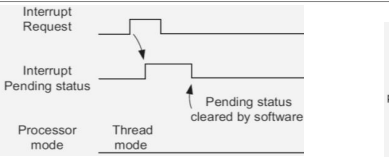
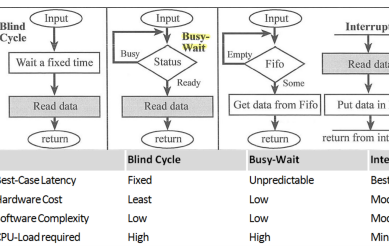


6.4 Preemption vs Subpriority

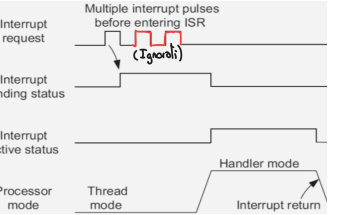
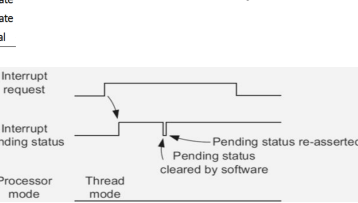
- **Cortex-M0+:** Supporta solo 2 bit di priorità utilizzabili solo come Preemption.
- **Cortex-M3/M4:** Tramite il *Priority Grouping*, gli 8 bit di priorità si dividono in:
 - **Preemption bits:** Decidono se un interrupt può interrompere fisicamente una ISR in esecuzione (*nesting*).
 - **Subpriority bits:** Criterio di spartizione temporale se due IRQ arrivano insieme

Exception Number	Vectors	CMSIS Interrupt Number	Memory Address
47	Interrupt#31 Vector	31	0x0000'00BC
...	...	...	...
18	Interrupt#2 Vector	2	0x0000'0048
17	Interrupt#1 Vector	1	0x0000'0044
16	Interrupt#0 Vector	0	0x0000'0040
15	SysTick Vector	-1	0x0000'003C
14	PendSV Vector	-2	0x0000'0038
13	Not Used	NA	0x0000'0034
12	Not Used	NA	0x0000'0030
11	SVC Vector	-5	0x0000'002C
10	Not Used	NA	0x0000'0028
9	Not Used	NA	0x0000'0024
8	Not Used	NA	0x0000'0020
7	Not Used	NA	0x0000'001C
6	Not Used	NA	0x0000'0018
5	Not Used	NA	0x0000'0014
4	Not Used	NA	0x0000'0010
3	Hard Fault Vector	-13	0x0000'000C
2	NMI Vector	-14	0x0000'0008
1	Reset Vector	NA	0x0000'0004
0	MSP Initial Value	NA	0x0000'0000

6.5 Soluzioni alternative & Segnali IRQ



1. **Software Clearing**
2. **Software Clearing mit Reset**
3. **ISR Reentry (Pulsed IRQ durante ISR)**
4. **Repulsed IRQ**
5. **Pulsed IRQ (Level stay active)**



7 Ottimizzazione e Strutture Dati

7.0.1 Switch Case

- **Pro:** Numero di cicli di clock definito e prevedibile.
- **Contro:** La condizione deve essere necessariamente una costante di tipo *int*.
- **Nota:** L'ordine dei case è rilevante per l'efficienza della ricerca.

7.0.2 Flusso di Controllo in Assembly

- **Selezioni (If/Else):** Si utilizza un salto in avanti con la **condizione invertita** rispetto al C (se la condizione falsa, salto oltre il blocco *then*).
- **Iterazioni (Loop):** Si utilizza un salto all'indietro alla fine del ciclo, mantenendo la **condizione assembly come in C** (salto all'inizio se la condizione è ancora vera).

7.0.3 Efficienza delle Strutture Dati

- **Array Locali:** Molto **inefficienti**. alto numero di cicli solo per l'inizializzazione nello stack frame

- **Struttura:** È consigliabile definire le strutture globalmente. L'inizializzazione locale su stack comporta un overhead elevato di cicli di clock.
- **Bitfields:** Estremamente inefficienti. Sebbene risparmino memoria, richiedono complesse operazioni di *Read-Modify-Write* (LDR, Shift, Mask, OR, STR)

8 Confronto Architetture ARM Cortex

Cortex-A (Application): Calcolo generico ad alte prestazioni (Linux/Android).

Cortex-R (Real-Time): Bassa latenza e alta affidabilità (automotive/medico).

Cortex-M (Microcontroller): Ridotto consumo energetico, interrupt deterministici.

Core	Thumb	Thumb-2	HW Mult.	HW Div.	Math Sat.	DSP	FPU	Arch
Cortex-M0	Mostly	Subset	1/32 cyc	No	No	No	No	ARMv6-M
Cortex-M0+	Mostly	Subset	1/32 cyc	No	No	No	No	ARMv6-M
Cortex-M1	Mostly	Subset	3/33 cyc	No	No	No	No	ARMv6-M
Cortex-M3	Fully	Fully	1 cyc	Yes	Yes	No	No	ARMv7-M
Cortex-M4	Fully	Fully	1 cyc	Yes	Yes	Yes	Optional	ARMv7E-M
Cortex-M7	Fully	Fully	1 cyc	Yes	Yes	Yes	Optional	ARMv7E-M

8.1 Evoluzione Cortex-M0 vs M0+

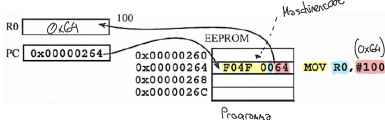
- **M0+ Pipeline:** Ridotta a 2 stadi per maggiore efficienza.
- **Accesso I/O:** Supporto per Single-Cycle GPIO.
- **Debugging:** Micro Trace Buffer (MTB) opzionale per il tracciamento dei salti.
- **MPU**

9 Adressierungsarten ARM (Cortex-M0)

- Register direkt
- Register Indirekt
- Register Indirekt mit offset [R?, #imm]
- Register Indirekt mit index [R?, R?]

9.1 Register Direkt, Immediate

- **Immediate:** MOV R0, #100 → Valore caricato direttamente.
- **Register:** MOVS R2, R1 → Copia valore tra registri.
- **Limitazioni:** Gli immediati sono codificati nell'istruzione (es. 8-bit per MOVS).

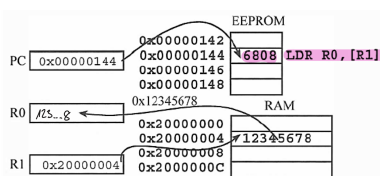


Pipeline

- + Es wird in jedem Cycle ein Befehl ausgeführt
- Bei sprüngen muss die Pipeline 'geleert' werden

9.2 Register Indirekt (ptr)

- **Semplice:** LDR R0, [R1] → R0 = valore all'indirizzo R1.
- **Con Offset:** LDR R0, [R1, #4] → R0 = valore all'indirizzo R1 + 4 (utile per array/struct).
- **Con Index:** LDR R0, [R1, R2] → R0 = valore all'indirizzo R1 + R2.



9.3 PC-Relative & Literal Pool

- **Problema:** Istruzioni da 32-bit non possono contenere costanti da 32-bit.
- **Soluzione:** L'assembler salva la costante nel **Literal Pool** (zona memoria vicina).
- **Accesso:** LDR R0, =0x12345678 viene tradotto in LDR R0, [PC, #offset].
- Richiedono sempre **due accessi** alla memoria

9.4 Registerlisten & Stack (SP-Relative)

- **Multiple:** LDM R7!, {R0-R3, R5} → Carica più registri partendo da indirizzo in R7.
- **Stack:** PUSH {R0-R2}, POP {R0-R2}.
- **Regola d'oro:** Il registro con numero più basso (R0) va sempre all'indirizzo di memoria più basso. L'ordine nelle graffe non conta per l'hardware.

9.5 Stack Framing

Lo Stack Frame è lo spazio riservato nella RAM per le variabili locali di una funzione.

- **Allocazione:** SUB SP, SP, #X riserva X byte (lo stack cresce verso indirizzi bassi).
- **Frame Pointer (R7):** Si usa MOV R7, SP per avere un riferimento fisso all'inizio del frame.
- **Accesso:** Le variabili vengono scritte/lette usando offset rispetto a R7, es: STR R0, [R7, #4].
- **Deallocazione:** Prima del ritorno (BX LR), si deve pulire lo stack con ADD SP, SP, #X.

C-Code	Assembly
void main(void)	main: ①
{	② SUB SP, SP, #8
uint32_t a = 100;	MOV R7, SP
int8_t b = -5;	MOVS R0, #100
// user code	MVNS R1, #4 (S - A; ind.)
	④ STR R0, [R7, #0]
	⑤ STRB R1, [R7, #4]
	; user code
	ADD SP, SP, #8
	BX LR

10 Il Processo di Compilazione

Il codice sorgente viene trasformato in binario attraverso diverse fasi:

- 1. Preprocessore:** Gestisce inclusioni e macro. (#define, #if, ...)
- 2. Compilatore:** Traduce il linguaggio ad alto livello in Assembly.
- 3. Assembler:** Converte Assembly in codice macchina rilocabile.
- 4. Linker/Binder:** Unisce file oggetto e librerie.
- 5. Loader:** Converte indirizzi rilocabili in assoluti e carica in memoria.

10.1 Analisi del Compilatore

- **Analisi Lessicale:** Rimozione spazi e raggruppamento token.
- **Analisi Sintattica/Semantica:** Controllo struttura e tipi di dati.

- **Ottimizzazione:** Miglioramento del codice per velocità o spazio.
- **Generazione:** Produzione del codice target.

11 Interfacciamento I/O

Le periferiche sono mappate in memoria (*Memory Mapped I/O*) tramite registri specifici:

- **Datenregister:** Contiene i dati da elaborare.
- **Steuerregister:** Configurazione della periferica.
- **Statusregister:** Indica lo stato corrente dell'I/O (es. Busy, Ready).

12 Esempi

```

1 ;Example: 64Bit Addition
2 LDR r4, =addrOp ;load address of int64 array [op1,op2,res]
3 LDM r4, {r0-r3} ;load op1[r1,r0] and op2[r2,r3]
4 ADDS r0, r0, r1 ;add lower words and set carry
5 ADCS r1, r2, r3 ;add higher words with carry in
6 ADDS r4, #8 ;set r4 to address of res
7 STM r4, {r0-r1} ;store 64bit result from r0, r1 to res

```

```

1 ; Example: 64Bit Subtraction
2 LDR r4, =addrOp ; load address of int64 array [op1,op2,res]
3 LDM r4, {r0-r3} ; load op1[r1,r0] and op2[r3,r2]
4 SUBS r0, r0, r2 ; sub lower words and set borrow
5 SBCS r1, r1, r3 ; sub higher words with borrow in
6 ADDS r4, #16 ; set r4 to address of res
7 STM r4!, {r0-r1} ; store 64bit result from r0, r1 to res

```

```

1 ;Example for 64bit logical shift left
2 LDR r2, =data64 ;load address of data64
3 LDM r2, {r0-r1} ;load data64 to r0, r1
4 LSLS r1, r1, #1 ;logical shift left r1 by 1
5 MOVS r3, #0 ;r3 = 0
6 LSLS r0, r0, #1 ;lsl r0 by 1 (carry = b31)
7 ADCS r1, r3 ;r1 = r1 + 0 + carry
8 STM r2!, {r0-r1} ;store r0,r1 in data64

```



Equivalente a: (·2)

Thumb-2 Assembly	C Source Code
<pre> 1 LDR R0, =G2 2 LDR R1, =G1 3 LDRH R0, [R0] ;R0 <- G2 4 SXTB R0, R0 5 LDR R1, [R1] ;R1 <- G1 6 CMP R0, R1 ;G2-G1 7 BGE else_if 8 then: 9 BL foo_one 10 B endif 11 else-if: 12 CMP R0, R1 13 BLE else 14 BL foo_two 15 B endif 16 else: 17 BL foo_three 18 endif: </pre>	<pre> 1 int32_t G1; // 32Bit 2 int16_t G2; // 16Bit 3 ... 4 if(G1 > G2) 5 { 6 foo_one(); 7 } 8 else if(G1 < G2) 9 { 10 foo_two(); 11 } 12 else 13 { 14 foo_three(); 15 } </pre>

Thumb-2 Assembly	C Source Code
<pre> 1 LDR R0, =G1 2 LDR R0, [R0] ;R0 <- G1 3 MOVS R1, #4 ;R1 = 4 4 CMN R0, R1 ;G1-(-4) 5 BLE else 6 CMP R0, R1 ;G1-4 7 BGT else 8 then: 9 BL foo_one 10 B endif 11 else: 12 BL foo_two 13 endif: </pre>	<pre> 1 int32_t G1; 2 ... 3 if((G1 > -4) && 4 (G1 <= 4)) 5 { 6 foo_one(); 7 } 8 else 9 { 10 foo_two(); 11 } </pre>

Thumb-2 Assembly	C Source Code
<pre> 0x080000f6 LDR R1, =State 0x080000f8 LDRB R0, [R1] ;R0 ← State 0x080000fa CMP R0, #3 ;Default State R0 - 3 0x080000fc BHI default ;R0 - 3 > 0 0x08000100 LSLS R0, R0, #2 ;R0 = R0 * 4 (offset) * 0x08000102 ADR R1, lookup ;R1 = 0x08000108 0x08000104 LDR R0, [R1, R0] ;R0 = val[lookup + offset] 0x08000106 MOV PC, R0 * * ;PC ← Addr of case lookup: 0x08000108 .word 0x08000130 ;addr of case 0 0x0800010c .word 0x08000136 ;addr of case 1 0x08000110 .word 0x0800013c ;addr of case 2 0x08000114 .word 0x08000142 ;addr of case 3 case_0: 0x08000130 BL foo_one 0x08000134 B end_switch case_1: 0x08000136 BL foo_two 0x0800013a B end_switch case_2: 0x0800013c BL foo_three 0x08000140 B end_switch case_3: 0x08000142 BL foo_four 0x08000146 B end_switch default: 0x08000148 MOVS R0, #0 0x0800014a STRB R0, [R1] end_switch: 0x0800014c - </pre>	<pre> uint_t State; // unsigned 8Bit ... switch(State) { case 0: foo_one(); break; case 1: foo_two(); break; case 2: foo_three(); break; case 3: foo_four(); break; default: State = 0; break; } ... </pre>

- 1. Controllo dei limiti (Bound Check):** Verifica iniziale del valore di State per saltare subito al default se l'indice è fuori range.
- 2. Calcolo dell'offset:** Moltiplicazione del valore per 4 ($State \ll 2$) tramite shift logico, poiché ogni puntatore nella look-up table occupa 32 bit (4 byte).
- 3. Recupero del puntatore:** Lettura dell'indirizzo di destinazione esatto sommando l'offset calcolato all'indirizzo base della tabella.
- 4. Salto (Branch):** Caricamento dell'indirizzo recuperato nel Program Counter (PC) per avviare l'esecuzione del caso specifico.